

Software Requirements Specification

Field	This example
Team name	Dek70
Project title	NUDTEE - Board Game Meetup Coordination App
SDLC chosen + reason	Agile (Scrum, feature-based sprints) — a 5-person team can divide work by functional requirement (FR-1 to FR-5), build and integrate in short iterative sprints, and adjust scope quickly given the limited build window.
Date / version	v1.0 · 2026-09-25
GitHub repo	https://github.com/6931503074-sudo/NUDTEE

1. Introduction

1.1 Purpose and scope

Board game players who want to find more people to play with especially people who enjoy the same genre currently rely on manual back-and-forth in LINE or Facebook groups. Existing groups do not show what genres each member enjoys, so organizers invite the wrong people or people who are not interested; nor is there a clear, live count of how many players a session still needs, so organizers must ask around one by one. This wastes time and often means a session never comes together even though enough interested players existed. NUDTEE is a small web app that lets a group show, in real time, which sessions are open, what genre each session is for, and exactly how many more players are needed so a session stops depending on someone manually counting replies in a chat thread. Success looks like this: an organizer opens the app, sees which sessions still need players and of what genre, and a member can join with one tap instead of a chat exchange.

1.2 Users and stakeholders

Name / role	Kind	What they need	What they fear / forbid
Player (primary user)	primary user	Find open sessions that match the genres they enjoy, and see live how many players are still needed	Being invited to sessions of a genre they dislike; joining a session that turns out to already be full
Session organizer	primary user	Fill a session with the right number of players who like the right genre, without asking one by one in chat	Ending up short of players because interested people never saw the post in time
Board game café / shop (indirect)	secondary stakeholder	More regular foot traffic from groups that successfully coordinate a visit	Being listed as a location without being able to control or confirm capacity
Hosting / backend provider (Supabase, Google, Vercel)	supplier	Usage that stays within free-tier limits during the course project	Sudden traffic spikes or misuse that break free-tier quotas

2. Overall description

2.1 Product context — what it does / does not

IN SCOPE (MVP this term)	OUT OF SCOPE (explicit promise)
• FR-1 Create a session (room) • FR-2 Set favorite game genres (profile tag) • FR-3 Join / leave a session • FR-4 Filter sessions by genre • FR-5 Real-time session status display	• Map / GPS-based location view • Waitlist when a session is full • Reminder / notification system • Player rating or review system • Integration with board game café/shop booking systems • In-app chat inside a session

2.2 Assumptions and constraints

- Assumption: every player has a smartphone or laptop and is willing to sign in with a Google account or phone number.
- Constraint: backend limited to Supabase free tier (500 MB database, 50,000 monthly active users, projects auto-pause after 7 days of inactivity).
- Constraint: hosting on Vercel/Netlify free tier; authentication via Google OAuth (no cost).
- Constraint: Thai and English UI text; no paid API or service anywhere in the MVP.
- Constraint: personal data stored is limited to display name, preferred genres, and session participation no chat content, no payment data.
- Constraint: 5-person team, shared build window with other coursework.

3. Functional requirements

FR-1 — Create a session (room)

User story:

As a player, I want to create a session with a genre, location, date/time, and desired number of players, so that others can see it and join.

Pain this traces to:

Sessions are currently announced informally in chat, with no structured, shared place to see what is being organized.

- Given a genre, location, date/time, and min/max player count entered, when the organizer saves it, then the session appears in the session list with those details.
- Given an existing session, when the organizer edits the details, then the change is shown immediately without creating a duplicate session.

FR-2 — Set favorite game genres (profile tag)

User story:

As a player, I want to tag the genres I enjoy on my profile, so that I can be matched with sessions I actually want to join.

Pain this traces to:

Members of existing chat groups do not know what genre each other enjoys, so organizers invite the wrong people or people who are not interested.

- Given a list of genres, when the player selects one or more, then those genres are saved to their profile and shown when relevant.

- Given a player removes a genre tag, when they save the change, then that genre no longer appears on their profile.

FR-3 — Join / leave a session

User story:

As a player, I want to join or leave an open session with one tap, so that the organizer does not need to count replies manually in chat.

Pain this traces to:

Organizers currently have no reliable running count of confirmed players and must ask one by one in a chat thread.

- Given an open session with available spots, when a player taps join, then they are added to the participant list and the remaining-spots count decreases by one.
- Given a session that is already full (current participants = max), when a player attempts to join, then the join is rejected and a full-session message is shown.

Decision logic in this FR:

This is the first FR with real decision logic: the system must compute remaining spots ($\text{max} - \text{current}$) and must reject a join attempt once the session is full, including handling two players tapping join at the same time when only one spot remains (only one join may succeed).

FR-4 — Filter sessions by genre

User story:

As a player, I want to filter the session list by genre, so that I only see sessions that match what I enjoy playing.

Pain this traces to:

Without a genre filter, players must read every session individually to find one matching their interest, which does not scale as more sessions are posted.

- Given at least one genre filter selected, when the player applies it, then only sessions matching that genre are shown.
- Given no filter selected, when the player views the list, then all open sessions are shown.

FR-5 — Real-time session status display

User story:

As a player or organizer, I want the session list to update live as people join or leave, so that the player counts I see are always accurate.

Pain this traces to:

Player counts discussed in chat quickly go stale as people change their mind, leading organizers to recount manually and repeatedly.

- Given another user joins or leaves a session, when the change is saved, then every other user viewing the session list sees the updated count within 2 seconds, without refreshing the page.
- Given a session becomes full through this real-time update, when other users view the list, then that session is visibly marked as full.

Decision logic in this FR:

This FR requires synchronizing state across multiple concurrently connected clients (via a real-time subscription channel) rather than simple add/remove of a single record — the golden-thread FR carried through below.

4. Non-functional requirements

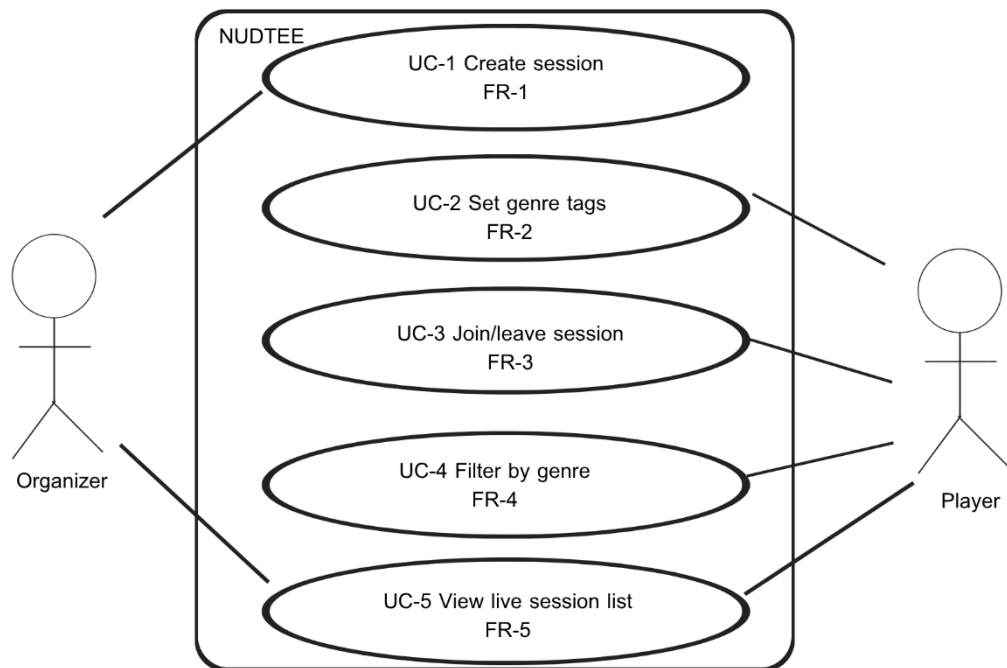
ID	Kind	Statement (testable)	How we will check
NFR-1	performance	The session list shall reflect a join/leave change on all connected clients within 2 seconds.	Two devices open the same session list; one joins, time until the other updates
NFR-2	usability	The app shall be usable on a mobile phone screen without horizontal scrolling on any core screen.	Manual check on a real phone for each of the 5 core screens
NFR-3	capacity	The system shall support at least 20 concurrently connected users viewing the real-time session list without errors.	Load test with 20 simulated concurrent connections
NFR-4	reliability	A session's remaining-spot count shall never go negative or exceed the stated maximum, even under concurrent join attempts.	Two clients tap join simultaneously on a session with one spot left; verify only one succeeds
NFR-5	security / privacy (required)	A user shall be able to edit or delete only sessions they created; attempting to modify another user's session shall fail.	Attempt to edit another account's session via the API directly and confirm it is rejected

5. Use cases

5.1 Use-case list

Use case	Actor	Related FR	Brief success story
UC-1 Create session	Organizer	FR-1	Organizer creates a "Catan, need 2 more" session; it appears in the list.
UC-2 Set genre tags	Player	FR-2	Player tags "Strategy" and "Party" on their profile.
UC-3 Join / leave session	Player	FR-3	Player joins a session; remaining-spot count drops from 2 to 1.
UC-4 Filter by genre	Player	FR-4	Player filters to "Strategy" and sees only matching sessions.
UC-5 View live session list	Player, Organizer	FR-5	Two players watch the same session; count updates live as a third joins.

5.2 Use-case diagram



6. Out of scope

1. Map / GPS-based location view.
2. Waitlist when a session is already full.
3. Reminder or notification system.
4. Player rating or review system.
5. Integration with board game café/shop booking systems.
6. In-app chat inside a session.

7. Traceability matrix (Golden Thread)

Problem / pain	Requirement (FR)	Solution / design	Feature built	Test
No structured place to announce sessions	FR-1 create session	(later)	(later)	(later)
Members' genre preferences unknown	FR-2 genre tag	(later)	(later)	(later)
No reliable running headcount, asked manually	FR-3 join/leave	(later)	(later)	(later)
Hard to find sessions matching one's genre	FR-4 genre filter	(later)	(later)	(later)
Player counts go stale between chat messages	FR-5 real-time display	Session entity + Supabase Realtime subscription channel	Live-updating session list	Two-client concurrent join/leave test (NFR-1, NFR-4)

8. AI usage log

Date	Tool model	What we asked	What AI produced	What we changed / verified	What we learned
Sept 2026	Claude	Whether a board game meetup app is a software-solvable problem; help scoping the problem, MVP, and FR list	Problem statement drafts, MVP scope table, FR-1 to FR-5 with logic analysis, SDLC comparison	We adjusted the MVP scope by explicitly moving complex features like GPS maps, waitlists, and in-app chats to the "Out of scope" section to ensure the 5-person team could meet the deadline under free-tier constraints.	We learned that adopting Agile and focusing only on the core pains (genre filtering and live headcount) is enough to build a usable product without overcomplicating it with booking systems.
